Unit 2	LINEAR DATA STRUCTURE

Syllabus	**Array:** Representation of arrays, Applications of arrays, sparse matrix and its representation **Stack:** Stack-Definitions & Concepts, Operations On Stacks, Applications of Stacks, Polish Expression, Reverse Polish Expression And Their Compilation, Recursion, Tower of Hanoi **Queue:** Representation Of Queue, Operations On Queue, Circular Queue, Priority Queue, Array representation of Priority Queue, Double Ended Queue, Applications of Queue **Linked List:** Singly Linked List, Doubly Linked list, Circular linked list, Linked implementation of Stack, Linked implementation of Queue, Applications of linked list.

Array

➢ What is an Array?

- An array is a data structure that contains a group of elements.
- Typically these elements are all of the same data type, such as an integer or string.
- Arrays are commonly used in computer programs to organize data so that a related set of values can be easily sorted or searched.
-
- An array is a collection of data items, all of the same type, accessed using a common name.

➢ Types of Array:

1. One Dimensional
2. Two Dimensional
3. Multi Dimensional

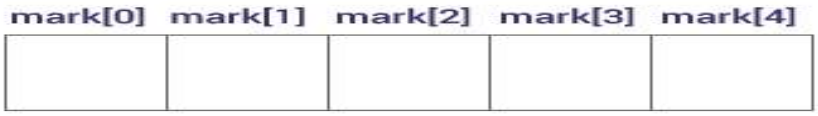
- **One Dimensional Array:**
  - An array which store its elements into a single row is called one dimensional array.

  - Syntax for Declare an Array

    datatype ArrayName[Arraysize];

    For Example: Float Mark[5];

- Here, we declared an array, mark, of floating-point type. And its size is 5. Meaning, it can hold 5 floating-point values.



- **How to Access Array Elements?**
  - You can access elements of an array by indices.
  - Suppose you declared an array mark as above. The first element is mark[0], the second element is mark[1] and so on.
  - Arrays have 0 as the first index, not 1. In our example, mark[0] is the first element.
  - If the size of an array is n, to access the last element, the n-1 index is used. In this example, mark[4].
  - Suppose the starting address of mark[0] is 2120. Then, the address of the mark[1] will be 2124. Similarly, the address of mark[2] will be 2128 and so on. This is because the size of a float is 4 bytes.

- **How to Initialize an Array?**
  - It is possible to initialize an array during declaration. For example,
    int mark[5] = {19, 10, 8, 17, 9};



  - Input and Output Array Element
  - Here's how you can take input from the user and store it in an array element.
    scanf("%d", &mark[2]); **// take input and store it in the 3rd element**
    scanf("%d", &mark[i-1]); **// take input and store it in the ith element**

  - Here's how you can print an individual element of an array.
    printf("%d", mark[0]); **// print the first element of the array**
    printf("%d", mark[2]); **// print the third element of the array**
    printf("%d", mark[i-1]); **// print ith element of the array**

  - **Example: Array Input/output**

```
#include<stdio.h>
void main()
{
    int a[4];
    int i, j;
```

```
    printf("Enter array element:");
    for(i = 0; i < 4; i++)
    {
        scanf("%d", &a[i]);    //Run
        time array initialization
    }
   printf("Array elements are:");
   for(j = 0; j < 4; j++)
    {
            printf("%d\n", a[j]);
    }
}
```

```
OUTPUT:
Enter array element:1
5
2
3

Array Elements are:1
5
2
3
```

- **Two Dimensional Array:**
  - C language supports multidimensional arrays also.
  - The simplest form of a multidimensional array is the two-dimensional array.
  - An array of arrays is known as 2D array.
  - The two dimensional (2D) array in C programming is also known as matrix.
  - A matrix can be represented as a table of rows and columns.
  - Two-dimensional arrays are declared as follows,

    data-type array-name[row-size][column-size] ;
    For Example, float x[3][4];

  - Here, x is a two-dimensional (2d) array. The array can hold 12 elements. You can think the array as a table with 3 rows and each row has 4 columns.

	Column 1	Column 2	Column 3	Column 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

  - Similarly, you can declare a three-dimensional (3d) array.
  - For example, **float y[2][4][3];**
  - Here, the array y can hold 24 elements.

- **How to initialize 2D Array?**
  - Different ways to initialize two-dimensional array:

    int c[2][3] = {{1, 3, 0}, {-1, 5, 9}};
    int c[][3] = {{1, 3, 0}, {-1, 5, 9}};
    int c[2][3] = {1, 3, 0, -1, 5, 9};

  - Runtime initialization of a two dimensional Array
  - Suppose array size is **a[3][4]**

```
for(i = 0; i < 3;i++)
{
        for(j = 0; j < 4; j++)
        {
        scanf("%d", &a[i][j]);
        }
}
```

```
i=0  j=0  : a[0][0]
     j=1  : a[0][1]
     j=2  : a[[0][2]
     j=3  : a[0][3]

i=1  j=0  :a[1][0]
     j=1  :a[1][1]
     j=2  :a[1][2]
     j=3  :a[1][3]

i=2  j=0  :a[2][0]
     j=1  :a[2][1]
     j=2  :a[2][2]
     j=3  :a[2][3]
i=3 condition false
```

- **Example: 2D Array Input/output**

```
#include<stdio.h>
void main()
{
int arr[3][4];
int i, j, k;
printf("Enter array element");
for(i = 0; i < 3;i++)
{
    for(j = 0; j < 4; j++)
    {
    scanf("%d", &arr[i][j]);
    }
}
for(i = 0; i < 3; i++)
{
    for(j = 0; j < 4; j++)
    {
        printf("%d", arr[i][j]);
    }
}
}
```

	0	1	2	3
0	5	10	15	20
1	25	30	35	40
2	45	50	55	60

Array Representation

➢ Array Representation

- The elements of a Two dimensional array can be arranged in two ways.
  1. Row major Representation
  2. Column major Representation

- **Row major Representation**:
  - If the elements are stored in row wise manner then it is called row major representation.
  - If we want to store element 10,20,30,40,50,60,70,80 and if size of array is a[3][4] then,

	0	1	2	3
0	10	20	30	40
1	50	60	70	80
2				

- **Column major Representation:**
  - If the elements are stored in column wise manner then it is called Column major representation.
  - If we want to store element 10,20,30,40,50,60,70,80 and if size of array is a[3][4] then,

	0	1	2	3
0	10	40	70	
1	20	50	80	
2	30	60		

➢ Address calculation of Array Elements

- You can calculate the address of any element which is stored in an array.
- To calculate the address of any element you must know,
  1. Base Address (Address of first element (which is at index 0).
  2. Data Types of Array (int(2bytes),Float(4 bytes)).
  3. Size of Array.
  4. Representation of array (Row major or Column Major)
  5. index of element for which you want to find address.

6. Formula.

- **Formula for Calculating Address**
  - **Row Major System:**
    The address of a location in Row Major System is calculated using the following formula:
    Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]
  - **Column Major System:**
    The address of a location in Column Major System is calculated using the following formula:
    Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]

    Where,
    B = Base address(Address of First Element)
    I = Row subscript of element whose address is to be found
    J = Column subscript of element whose address is to be found
    W = Storage Size of one element stored in the array (in byte) (Int=2 byte,Float=4 byte)
    Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)
    Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)
    M = Number of row of the given matrix
    N = Number of column of the given matrix

- **Example 1: Consider an integer array int a[3][4].if the base address is 1050,find the address of the element a[2][2] with row major and column major representation of array.**

  - Row Major System:
    The address of a location in Row Major System is calculated using the following formula:
    Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]

    B = Base address(Address of First Element)=1050
    I = Row subscript of element whose address is to be found=2
    J = Column subscript of element whose address is to be found=2
    W = Storage Size of one element stored in the array =2 bytes(because datatype is int).
    Lr = Lower limit of row/start row index of matrix, if not given assume 0 (zero)=0

Lc = Lower limit of column/start column index of matrix, if not given assume 0 (zero)=0
M = Number of row of the given matrix=3
N = Number of column of the given matrix=4

Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]
A [2][2] = 1050 + 2 * [4 * (2 – 0) + (2 – 0)]
= 1070

	0	1	2	3
0	1050	1052	1054	1056
1	1058	1060	1062	1064
2	1066	1068	?(1070)	

- Column Major System:
  The address of a location in Column Major System is calculated using the following formula:
  Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]

  A [2][2] = 1050 + 2 * [(2 – 0) + 3*(2 – 0)] = 1050+16 = 1066

	0	1	2	3
0	1050	1056	1062	
1	1052	1058	1064	
2	1054	1060	?(1066)	

- **Example 2: Example 2:A 2-D array defined as A[r, c] where 1 ≤ r ≤ 4, 5≤ c ≤ 8, requires 2 bytes of memory for each element. If the array is stored in Row-major order form and Column-major order form, calculate the address of A[3,7] given the Base address as 2000**

  Size of Array A[1:4][5:8]
  B=2000, W=2 byte, I=3, J=7, Lr=1, Lc=5, M=4, N= 4

  - Row Major System:
    The address of a location in Row Major System is calculated using the following formula:
    Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]

=2000 + 2[4*(3-1) +(7-5)
=2000 + 2*10
=2000+20
=2020

	5	6	7	8
1	2000	20002	2004	2006
2	2008	2010	2012	2014
3	2016	2018	?(2020)	
4				

- Column Major System:
  The address of a location in Column Major System is calculated using the following formula:
  Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]
  =2000 + 2[(3-1) +4*(7-5)
  =2000 + 2*10
  =2000+20
  =2020

	5	6	7	8
1	2000	2008	2016	
2	2002	2010	2018	
3	2004	2012	?(2020)	
4	2006	2014		

- **Example 3:Given a two dimensional array Z1(2:9, 9:18) stored in column-major order with base address 100 and size of each element is 4 bytes, find address of the element Z1(4, 12).**

  Size of Array A[2:9][9:18]
  B=100, W=4 byte, I=4, J=12, Lr=2, Lc=9, M=8(9-2+1), N= 10(18-9+1)

  - Column Major System:
    The address of a location in Column Major System is calculated using the following formula:

Address of A [I][J] Column Major Wise = B + W * [(I – Lr) + M * (J – Lc)]
=100 + 4[(4-2) +8*(12-9)
=100 + 4[2+8*3]
=100+4*26
=100+104
=204

	9	10	11	12	13	14	15	16	17	18
2	100	132	164	196						
3	104	136	168	200						
4	108	140	172	?204						
5	112	144	176							
6	116	148	180							
7	120	152	184							
8	124	156	188							
9	128	160	192							

- Row Major System:
  The address of a location in Row Major System is calculated using the following formula:
  Address of A [I][J] = B + W * [N * (I – Lr) + (J – Lc)]
  =100 + 4[10(4-2) +(12-9)
  =100 + 4[20+3]
  =100+4*23
  =100+92
  =192

	9	10	11	12	13	14	15	16	17	18
2	100	104	108	112	116	120	124	128	132	136
3	140	144	148	152	156	160	164	168	172	176
4	180	184	188	?192						
5										
6										
7										
8										
9										

Applications of Array

➢ Applications

- Array are used for representing some sequential data structures such as stack and queues.
- Array are used to represent sparse matrices.
- Array are widely used to implement mathematical vector, matrices and other kind of tables.
- Array can be used for sorting elements in ascending or descending order.
- It is used in every possible situation where you need to gather similar objects at one place.
- Arrays are used to implement Search Algorithms.
- Arrays are also used to implement CPU Scheduling Algorithms.

Sparse Matrix and its Representation

➢ Sparse Matrix

- Sparse matrix is a matrix which contains very few non-zero elements.
- A matrix that is not sparse is called dense matrix.
- Normally matrices are represent in a two dimensional array.
- In a matrix if there are m rows and n columns then the space required to store the numbers will be m*n*s where s is the number of bytes required to store the value.
- Suppose, there are 10 rows and 10 columns and we have to store the integer values then space complexity will be 10*10*2 =200 bytes.
- For example, if the matrix is of size 10*10 and only 10 elements are nonzero. Then for accessing 10 elements one has to make 100 times scan.
- Also only 10 spaces will be with non-zero elements, remaining spaces of matrix will be filled with zeros only.
- The concept of sparse matrix has therefore came forward to investigate that representation will store only non-zero elements of the matrix and still carry out the operations quite efficiently.

➢ Sparse Matrix Representation

- Representation of sparse matrix will be a triplet only.
- Sparse matrix means very few non-zero elements having in it.
- Rest of the spaces are having the values zero which are basically useless values.

- So in this efficient representation we will consider all the non-zero values along with their positions.
- For example –Suppose a matrix is 6 * 7 and the number of non-zero values are say 8.

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	-10
1	55	0	0	0	0	0	0
2	0	0	0	0	0	-23	0
3	0	67	0	0	0	0	88
4	0	0	0	14	-28	0	0
5	99	0	0	0	0	0	0

- In normal representation of the matrix this will take 6 * 7 *2 =84 bytes.
- In row wise representation of sparse matrix the 0th rows will store total rows of the matrix, total column of the matrix, and the total non-zero values.

index	Row No	Column No	Value
0	6	7	8
1	0	6	-10
2	1	0	55
3	2	5	-23
4	3	1	67
5	3	6	88
6	4	3	14
7	4	4	-28
8	5	0	99

- In normal representation of the matrix this will take 6 * 7 *2 =84 bytes.
- In sparse matrix representation (total no of non-zero values +1) * 3 * 2 bytes.
- So it will be (8 + 1) * 3 * 2 = 54 Bytes.
- Clearly in normal representation 30 bytes is unused space, which are saving in the sparse matrix representation.

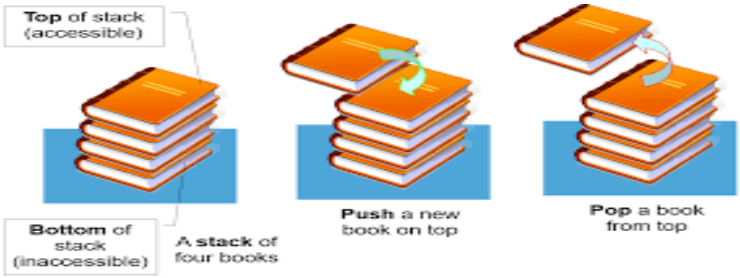
Stack Definition and Concept

➢ Stack Definition and Concept

- A linear list which allow insertion and deletion of an element at one end only is called Stack.
- A Stack is a data structure which is used to store data in a particular order.
- It follows Last In First Out (LIFO) Order.

- The element which is inserted last, that will be come out first.
- There are many real-life examples of a stack.
- Consider an example of plates stacked over one another in the canteen.

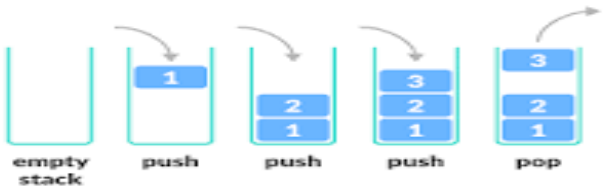




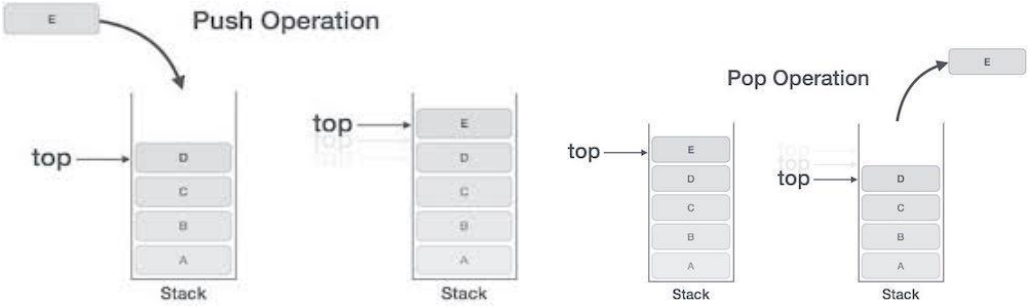
Operations on Stack

There are four operations which we can perform on stack.

- Push (Insert): Which insert an element at TOP of the stack.
- Pop(Delete): Which Remove the most recently added elements from the TOP of the stack.
- Peep (Search): To find ith element from TOP of the stack.
- Change (Update): Changes the value of the ith element from the TOP of the stack.



- TOP is a pointer that points to the top element of stack.
- Initially when the Stack is Empty, TOP has a value of “Zero”.
- Each time a new element is inserted in the stack, the pointer is Incremented by “one”.
- The pointer is decremented by “one” each time a deletion is made from the stack.



➢ Procedure: PUSH(S,TOP,X)

- This procedure insert an element X to the top of a stack.
- Stack is represented by a vector S Containing N Elements.
- A pointer TOP represents the top element of the stack.

1. [Check for Stack overflow]
   If Top ≥ N
   then Write ('Stack Overflow')
   Return

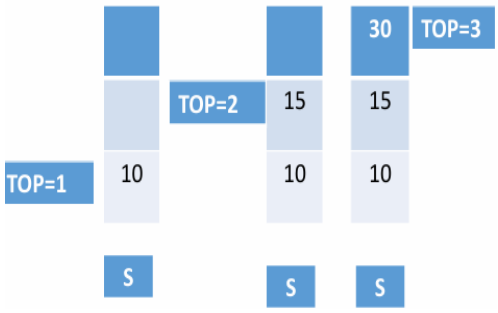
2. [Increment Top]
   Top ← Top + 1

3. [Insert Element]
   S[Top] ← X

4. [Finished]
   Return

Stack is Empty, TOP=0
N=3

PUSH(S,TOP,10) ,TOP=1
PUSH(S,TOP,15),TOP=2
PUSH(S,TOP,30),TOP=3
PUSH(S,TOP,40),
Overflow



➢ Procedure: POP(S,TOP)

- This procedure remove and returns the top element from a stack.
- Stack is represented by a vector **S** Containing **N** Elements.
- A pointer **TOP** represents the top element of the stack.

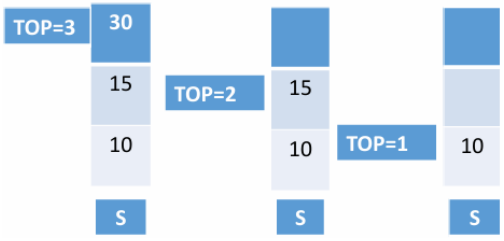
1. [Check for underflow on Stack]
   If Top = 0
   then Write ('Stack Underflow on Pop')
   Exit

2. [Decrement Pointer]
   Top ← Top-1

3. [Return former top element of Stack]
   Return (S[Top+1])

POP(S,TOP),TOP=3,
TOP=2
Return S[2+1]
POP(S,TOP),TOP=2,
TOP=1
Return S[1+1]
POP(S,TOP),TOP=1
TOP=0
Return S[0+1]
POP(S,TOP),TOP=0
Underflow

Stack is FULL, TOP=3,
N=3

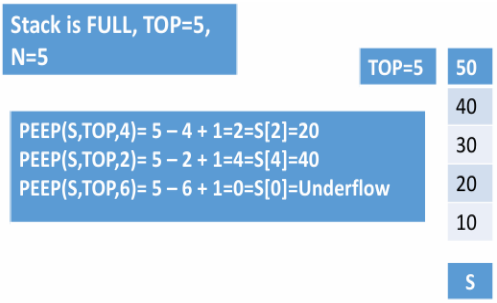


➢ **Procedure: PEEP(S,TOP,I)**

- This procedure returns the value of ith element from top of the stack.
- The element is not deleted by this function.
- Stack is represented by a vector S Containing N Elements.
- A pointer TOP represents the top element of the stack.

1. [Check for Stack Underflow]
   If (Top - I + 1) ≤ 0
   then Write ('Stack Underflow on Peep')
   Exit

2. [Return Ith element from top of the stack]
   Return (S[Top-I+1])



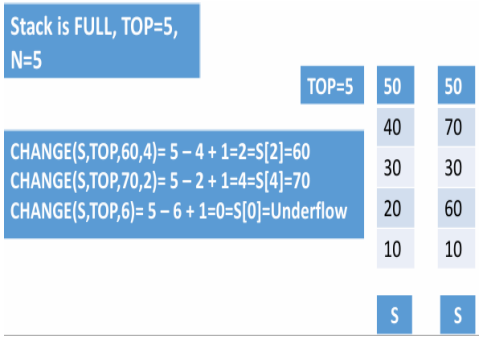
➢ **Procedure: CHANGE(S,TOP,X,I)**

- This procedure changes the value of ith element from the top of the stack to the value contained in X.
- Stack is represented by a vector S Containing N Elements.
- A pointer TOP represents the top element of the stack

1. [Check for stack underflow]
   If (Top - I + 1) ≤ 0
   then Write ('Stack Underflow on Change')
   Exit

2. [Change the Ith element from top of stack]
   S[Top - I + 1] ← X

3. [Finished]
   Return



Application of Stack

- **Expression Conversion**: An expression is a collection of operators and operands that represents a specific value.
- In above definition, operator is a symbol which performs a particular task like arithmetic operation.
- Operands are the values on which the operators can perform the task.
- For Example, A + B * C

- An arithmetic expression can be written in three different but equivalent notations, i.e., without changing the essence or output of an expression.
- These notations are –
    1. Infix Notation
    2. Prefix (Polish) Notation
    3. Postfix (Reverse-Polish) Notation

- **Infix Notation**: We write expression in infix notation, e.g. a + b, where operators are used in-between operands.

- **Prefix Notation**: In this notation, operator is prefixed to operands, i.e. operator is written ahead of operands. For example, +ab. This is equivalent to its infix notation a + b. Prefix notation is also known as Polish Notation.

- **Postfix Notation**: This notation style is known as Reversed Polish Notation (RPN). In this notation style, the operator is post fixed to the operands i.e., the operator is written after the operands. For example, ab+. This is equivalent to its infix notation a + b.

    Infix = a + b
    Prefix = +ab
    Postfix = ab+

- **Precedence and Associativity:**
    - When an operand is in between two different operators, which operator will take the operand first, is decided by the **precedence** of an operator over others. For example –

      a + b * c ⇨ a + (b * c)

    - As multiplication operation has precedence over addition, b * c will be evaluated first.
    - **Associativity** describes the rule where operators with the same precedence appear in an expression.
    - For example, in expression a + b − c, both + and – have the same precedence, then which part of the expression will be evaluated first, is determined by associativity of those operators.
    - Here, both + and − are left associative, so the expression will be evaluated as **(a + b) − c**.
    - Precedence and associativity determines the order of evaluation of an expression. Following is an operator precedence and associativity table (highest to lowest) –

Sr.No.	Operator	Precedence	Associativity
1	Exponentiation ^	Highest	Right Associative
2	Multiplication (*) & Division (/)	Second Highest	Left Associative
3	Addition (+) & Subtraction (–)	Lowest	Left Associative

- **Infix to postfix conversion**: There are two algorithms for that.

1. UNPARENTHESIZED_SUFFIX(Without Bracket).
   e.g. a + b * c - d / e

2. REVPOL(With Bracket)
   e.g. a * (b + c) – d

- **Algorithm UNPARENTHESIZED_SUFFIX**

This algorithm converts the string INFIX to its reverse Polish string equivalent.
INFIX: Input string INFIX representing an infix expression whose single character symbols have precedence values and ranks as given in Table.
S: vector S representing a stack.
NEXTCHAR: String function NEXTCHAR which, when invoked, returns the next character of the input string.
RANK: RANK contains the value of each head of the reverse Polish string.
NEXT: NEXT contains the symbol being examined.
TEMP:TEMP is a temporary variable which contains the unstacked element.
POLISH: which contain final output string(postfix form)
We assume that the given input string is padded on the right with the special symbol "#".

Symbol	Precedence (f)	Rank (r)
+, -	1	-1
*, /	2	-1
a, b, c, ... (Variables)	3	1
#	0	--

Algorithm UNPARENTHESIZED_SUFFIX:

1. [Initialize the stack]
   Top ← 1
   S[Top] ← '#'

2. [Initialize output string and rank count]
   POLISH ← ''
   RANK ← 0

3. [Get first input symbol]
   NEXT ← NEXTCHAR(INFIX)

4. [Translate the infix expression]
   Repeat thru step 6 while NEXT ≠ '#'

5. [Remove symbols with greater or equal precedence from stack]
   Repeat while f(NEXT) ≤ f(S[Top])
   TEMP ← Pop(S, Top) (this copies the stack contents into TEMP)
   POLISH ← POLISH O TEMP
   RANK ← RANK + r(TEMP)
   If RANK < 1
   then Write ('INVALID')
      Exit

6. [Push current symbol onto stack and obtain next input symbol]
   Call Push (S, Top, NEXT)
   NEXT ← NEXTCHAR (INFIX)

7. [Remove remaining elements from stack]
   Repeat while S[Top] ≠ '#'
   TEMP ← Pop (S, Top)
   POLISH ← POLISH O TEMP
   RANK ← RANK + r(TEMP)
   If RANK < 1
   then Write ('INVALID')
   Exit

8. [Is the expression valid?]
   If RANK = 1
   then Write ('VALID')
   else Write ('INVALID')
      Exit

- **Example: A + B * C –D**

  INFIX: A + B * C –D #

Input Symbol (NEXT)	Content of Stack	Reverse Polish Expression(POLISH)	Rank
	#		0
A (3≤0)	# A		0
+ (1≤3) (1≤0)	# +	A	1
B (3≤1)	# + B	A	1
* (2≤3) (2≤1)	# + *	AB	2
C (3≤2)	# + * C	A B	2
- (1≤3) (1≤2) (1≤1)	# -	A B C * +	1
D (3≤1)	# - D	A B C * +	1
#	#	**A B C * + D -**	1

Symbol	Precedence (f)	Rank (r)
+, -	1	-1
*, /	2	-1
a, b, c, ... (Variables)	3	1
#	0	--

- **Algorithm REVPOL:**

- This algorithm converts INFIX into reverse polish and places the result into POLISH.
- Given an input string INFIX containing an infix expression which has been padded on the right with ‘)’.
- All Symbol have precedence value given in table.
- Stack is represented by vector S, TOP denotes the top of the stack, Algorithm PUSH and POP are used for stack manipulation.
- Function NEXTCHAR returns the next symbol from given input string.
- The integer variable RANK contains the rank of Expression.
- The string variable TEMP is used for temporary storage purpose.

Symbol	Input Precedence Function f	Stack Precedence Function g	Rank Function r
+, -	1	2	-1
*, /	3	4	-1
↑	6	5	-1
a, b, c, ... (Variables)	7	8	1
(	9	0	--
)	0	--	--

1. [Initialize stack]
   Top ← 1
   S[Top] ← ‘(‘

2. [Initialize output string and rank count]
   POLISH ← ‘’
   RANK ← 0

3. [Get first input symbol]
NEXT ← NEXTCHAR (INFIX)

4. [Translate the infix expression]
   Repeat thru step 7 while NEXT ≠ ‘’

5. [Remove symbols with greater precedence from stack]
   If Top < 1
   then Write (‘INVALID’)
   Exit
   Repeat while f(NEXT) < g(S[TOP])
   TEMP ← Pop (S, Top)
   POLISH ← POLISH O TEMP
   RANK ← RANK + r(TEMP)
   If RANK < 1
   then Write (‘INVALID’)
   Exit

6. [Are there matching parentheses?]
   If f(NEXT) ≠ g(S[Top])
   then Call Push (S, Top, NEXT)
   else Pop (S, Top)

7. [Get next input symbol]
   NEXT ← NEXTCHAR (INFIX)

8. [Is the expression valid?]
   If Top ≠ 0 or RANK ≠ 1
   then Write (‘INVALID’)
   else Write (‘VALID’)
      Exit

- **Example: A *(B + C) –D**
  INFIX: (A *(B + C) –D)

Input Symbol (NEXT)	Content of Stack	Reverse Polish Expression(POLISH)	Rank
	(		0
A (7<0)	(A		
* (3<8) (3<0)	(*	A	1
((9<4)	(* (	A	1
B (7<0)	(* (B	A	1
+ (1<8) (1<0)	(* (+	A B	2
C (7<2)	(* (+ C	A B	2
) (0<8) (0<2) (0<0)	(*	A B C +	2
- (1<4) (1<0)	(-	A B C + *	1
D (7<2)	(- D	A B C + *	1
) (0<8) (0<2) (0<0)		**A B C + * D -**	1

- **Algorithm Evaluate_Postfix**

- Given an input string POSTFIX represent postfix expression.
- This algorithm evaluates postfix expression and put the result into variable Answer.
- Stack is represent by vector S, TOP denotes the top of the stack, Algorithm PUSH and POP are used for stack manipulation.
- Function NEXTCHAR returns the next symbol from given input string.
- OP1, OP2 and TEMP are used for temporary storage purpose.
- PERFORM_OPERATION is a function which perform required operation on OP1 and OP2.

1. [Initialize Stack]
   Top ← 0
   Answer← 0

2. [Evaluate the postfix expression]
   Repeat until last character
   TEMP← NEXTCHAR (POSTFIX)
   If TEMP is OPERAND
   Then PUSH(S,TOP,TEMP)
   Else
   OP2 ← POP(S,TOP)
   OP1 ← POP(S,TOP)

Answer ← PERFORM_OPERATION(OP1,OP2,TEMP)
PUSH(S, TOP, Answer)

3. [Return Answer from Stack]
Return (POP(S,TOP))

- **Algorithm Evaluate_Postfix Example**

POSTFIX: 9 3 4 * 8 + 4 / -

TEMP	OP1	OP2	Answer	Stack
9				9
3				9,3
4				9,3,4
*	3	4	12	9,12
8				9,12,8
+	12	8	20	9,20
4				9,20,4
/	20	4	5	9,5
-	9	5	4	**4**

POSTFIX: 5 4 6 + * 4 9 3 / + *

TEMP	OP1	OP2	Answer	Stack
5				5
4				5,4
6				5,4,6
+	4	6	10	5,10
*	5	10	50	50
4				50,4
9				50,4,9
3				50,4,9,3
/	9	3	3	50,4,3
+	4	3	7	50,7
*	50	7	350	**350**

- **Algorithm Evaluate_Prefix**

- Given an input string PREFIX represent prefix expression.
- This algorithm evaluates prefix expression and put the result into variable Answer.
- Stack is represent by vector S, TOP denotes the top of the stack, Algorithm PUSH and POP are used for stack manipulation.
- Function NEXTCHAR returns the next symbol from given input string.
- OP1,OP2and TEMPare used for temporary storage purpose.
- PERFORM_OPERATION is a function which perform required operation on OP1 and OP2.

1. [Initialize Stack]
   Top ← 0
   Answer← 0

2. [Evaluate the prefix expression]
   Repeat from last character up to First
   TEMP← NEXTCHAR (PREFIX)
   If TEMP is OPERAND
   Then PUSH(S,TOP,TEMP)
   Else
   OP1 ← POP(S,TOP)
   OP2← POP(S,TOP)
   Answer ← PERFORM_OPERATION(OP1,OP2,TEMP)
   PUSH(S, TOP, Answer)

3.[Return Answer from Stack]
   Return (POP(S,TOP))

- **Example 1:Prefix Evaluation**

  PREFIX: -9 / + * 3 4 8 4

TEMP	OP1	OP2	Answer	Stack
4				4
8				4,8
4				4,8,4
3				4,8,4,3
*	3	4	12	4,8,12
+	12	8	20	4,20
/	20	4	5	5
9				5,9
-	9	5	4	**4**

- **Example 2: Prefix Evaluation**

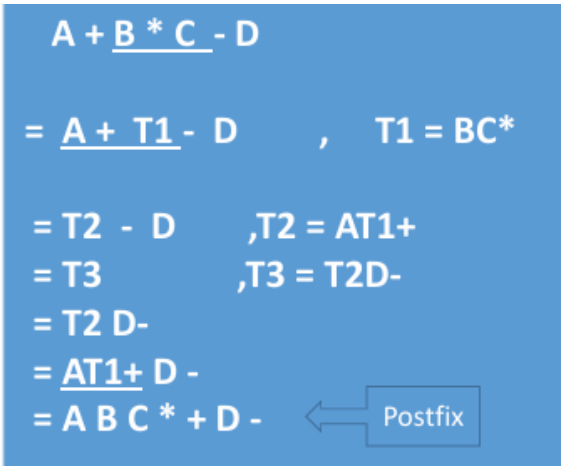
  PREFIX: + * A B –C + C * B A **(A= 4, B=8, C=12)**

  + * 48 –12+ 12* 8 4

TEMP	OP1	OP2	Answer	Stack
4				4
8				4,8
*	8	4	32	32
12				32,12
+	12	32	44	44
12				44,12
-	12	32	-20	-20
8				-20,8
4				-20,8,4
*	4	8	32	-20,32
+	32	-20	12	**12**

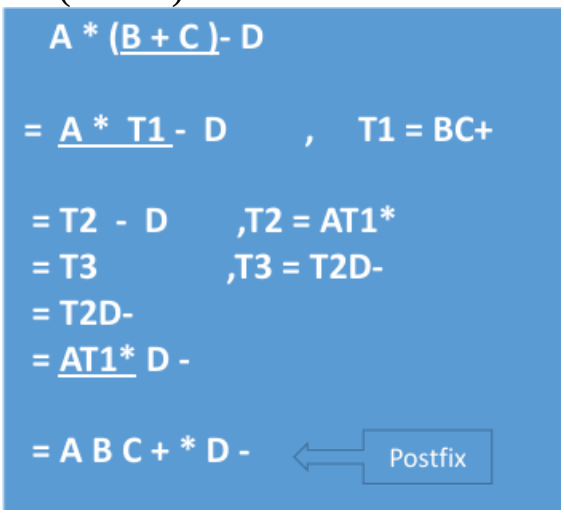
- **Example: infix to postfix**

A + B * C –D ←Infix



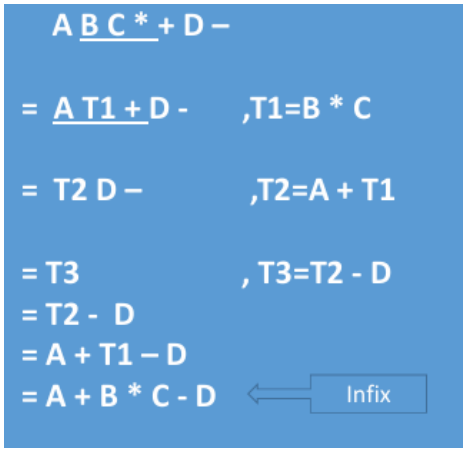
- **Example: Infix to postfix**

A *(B + C) –D ←Infix



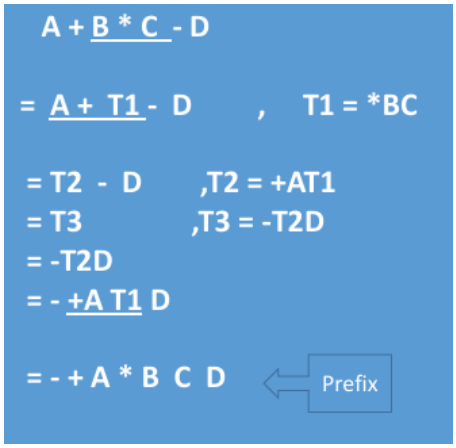
- **Example: postfix to infix**

A B C * + D - ←Postfix



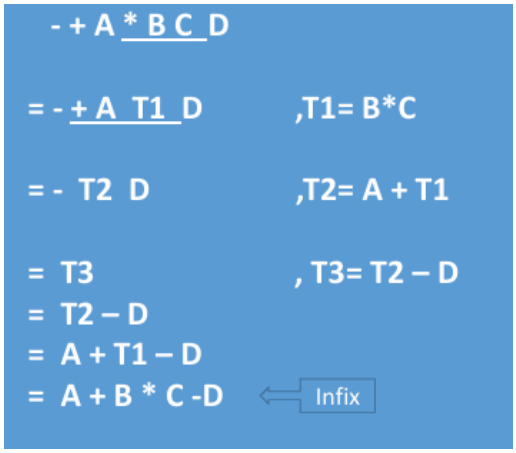
- **Example: infix to prefix**

A + B * C –D ←Infix



- **Example: prefix to infix**

- + A * B C D ←Prefix



- **Example: Transform the following expression to postfix (reverse polish) and evaluate postfix expression by assuming A=1, B=2,C=3, D=4, E=6, F=6, G=1, I=3 and J=3**

A + B -C *D / E +F $ G / (I+J) ←Infix

```
  A + B - C * D / E + F $ G / (I+J)
=  A + B - C * D / E + F $ G / T1      T1= I J +
=  A + B - C * D / E + T2 / T1        T2= F G $
=  A + B - T3 / E + T2 / T1           T3=C D *
=  A + B - T4 + T2 / T1               T4=T3 E /
=  A + B - T4 + T5                    T5=T2 T1 /
=  T6 - T4 + T5                       T6= A B +
=  T7 + T5                            T7= T6 T4 –
=  T8                                 T8= T7 T5 +
=  T7 T5 +
=  T6 T4 – T5 +
=  A B + T4 – T5 +
=  A B + T4 – T2T1/ +
=  A B + T3E/ - T2 T1 / +
=  A B + CD* E / - T2 T1 /+
=  A B + C D * E / - FG$ T1 / +
=  A B + C D * E / - F G $ IJ+ / +
=  A B + C D * E / - F G $ I J + / +
```

Postfix

Recursion

➢ What is Recursion?

- The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.
- A recursive algorithm takes one step toward solution and then recursively call itself to further move.
- The algorithm stops once we reach the solution.
- Since called function may further call itself, this process might continue forever.
- So it is essential to provide a base case to terminate this recursion process.

➢ Steps to Implement Recursion

- Recursion, in programming, involves a function calling itself to solve a problem.
- The process can be broken down into distinct steps:

- **Base Case Definition**:
  - Identify the simplest instance of the problem that can be solved directly without further recursive calls.
  - This is the stopping condition for the recursion.

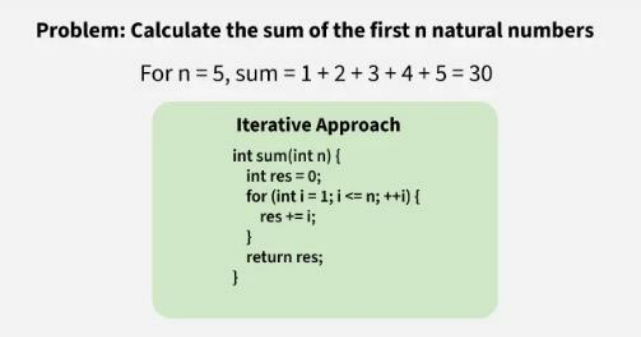
- Without a base case, the function would call itself infinitely, leading to a stack overflow.

- **Recursive Case Definition**:
  - Express the larger problem in terms of one or more smaller, similar subproblems.
  - The function calls itself with these smaller inputs, effectively breaking down the original problem into manageable parts.

- **Work Towards the Base Case**:
  - Each recursive call must bring the problem closer to the base case.
  - This means the input to the recursive call should be a smaller or simpler version of the original input.

- **Combining Subproblem Solutions:**
  - After the recursive calls return their solutions for the sub problems, combine these results to construct the solution for the original, larger problem.
  - This step is particularly relevant in problems like tree traversals or sorting algorithms.

- **Example : Sum of Natural Numbers**
  - Let us consider a problem to find the sum of natural numbers, there are several ways of doing that but the simplest approach is simply to add the numbers starting from 0 to n.



- **Need of Recursion?**
  - Recursion helps in logic building.

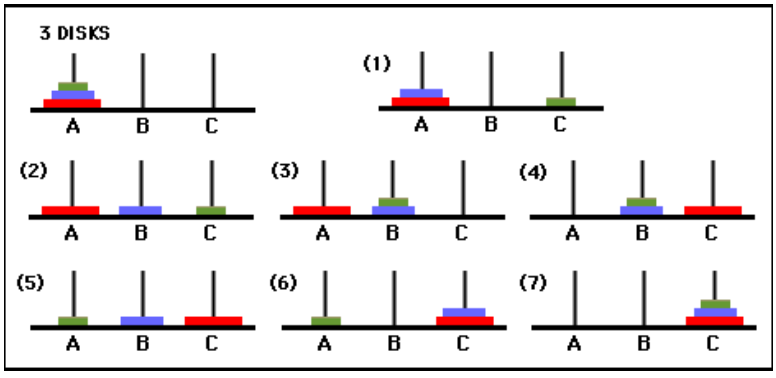
- Recursive thinking helps in solving complex problems by breaking them into smaller sub problems.
- Recursive solutions work as a basis for Dynamic Programming and Divide and Conquer algorithms.
- Certain problems can be solved quite easily using recursion like Towers of Hanoi (TOH), Inorder/Preorder/Postorder Tree Traversals, DFS of Graph, etc.

Tower of Hanoi

➢ Tower of Hanoi

- The Tower of Hanoi is a mathematical game or Puzzle.
- It consists of three rods and a number of disks of different sizes, which can slide onto any rod.
- The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape.
- The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:
  1. Only one disk can be moved at a time.
  2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
  3. No larger disk may be placed on top of a smaller disk.
- With 3 disks, the puzzle can be solved in 7 moves. The minimal number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where n is the number of disks.
- The idea is to use the helper node to reach the destination using recursion. Below is the pattern for this problem:

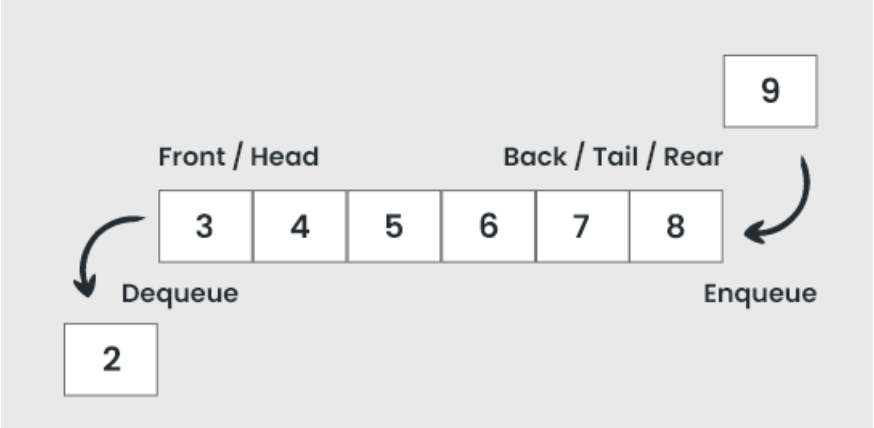
  1. Shift 'N-1' disks from 'A' to 'B', using C.
  2. Shift last disk from 'A' to 'C'.
  3. Shift 'N-1' disks from 'B' to 'C', using A.



Queue

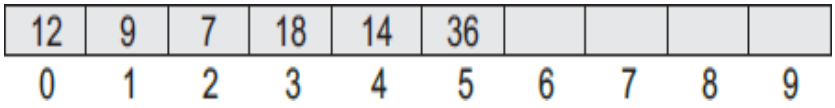
➢ Queue Definition and Concept

- A linear list which permits **deletion** to be performed **at one end** of the list and **insertion at the other end** is called queue.
- The information in such a list is processed **FIFO (first in first out) or FCFS (first come first served)** manner.
- **Front** is the end of queue from that deletion is to be performed.
- **Rear** is the end of queue at which new element is to be inserted.
- Insertion operation is called **Enqueue** & deletion operation is called **Dequeue**.

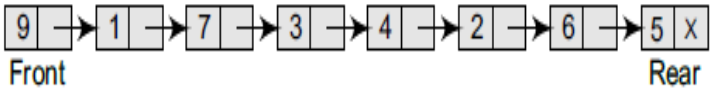


Representation of Queue

- **Array Representation of Queues**



- **Linked Representation of Queue**

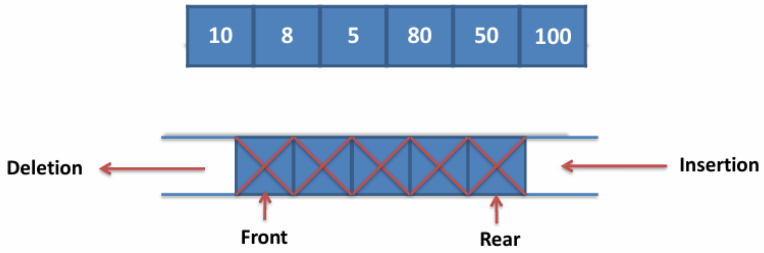


- **Types of Queue**
  - Simple Queue.
  - Circular Queue
  - Priority Queue
  - Double Ended Queue

Operations on Simple Queue

➢ What is a Simple Queue?

- **Insert**: Insert an element at **Rear** End.



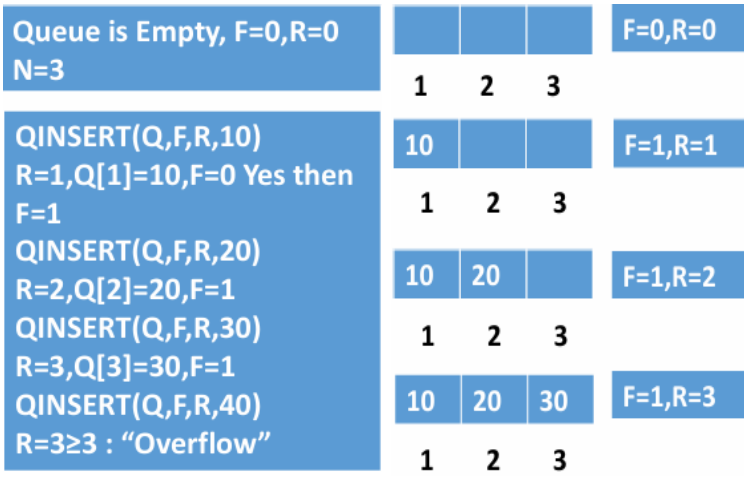
QINSERT (Q, F, R, N, Y)

- This procedure inserts Y at rear end of Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

1. [Overflow?]
   If R ≥ N
   Then Write ('Overflow')
   Return

2. [Increment rear pointer]
   R ← R + 1

3. [Insert element]
   Q[R] ← Y



4. [Is front pointer properly set?]
   If F = 0
   then F ← 1
   Return

- **Delete:** Delete an Element from **Front** End**.**

QDELETE (Q, F, R)

- This function deletes & returns an element from front end of the Queue.
- Queue is represented by a vector Q containing N elements.

- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

1. [Underflow?]
   If F = 0
   then Write ('Underflow')
   Return (0) ('0' denotes an empty queue)

2. [Delete element]
   Y ← Q[F]

3. [Queue empty?]
   If F = R
   then F ← R ← 0
   else F ← F + 1 (Increment front pointer)

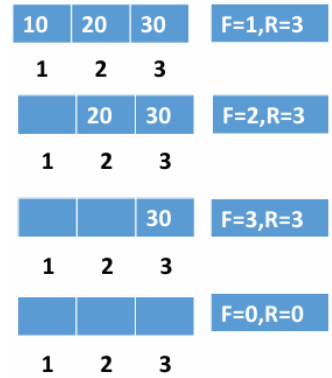
4. [Return element]
   Return (Y)

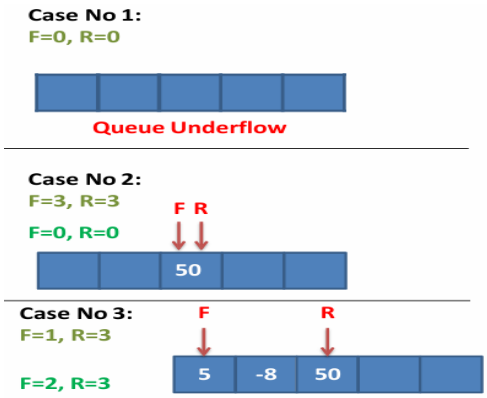
Queue is FULL, F=1,R=3 N=3

QDELETE(Q,F,R) ,F=1
Y=Q[1],Y=10,F=2

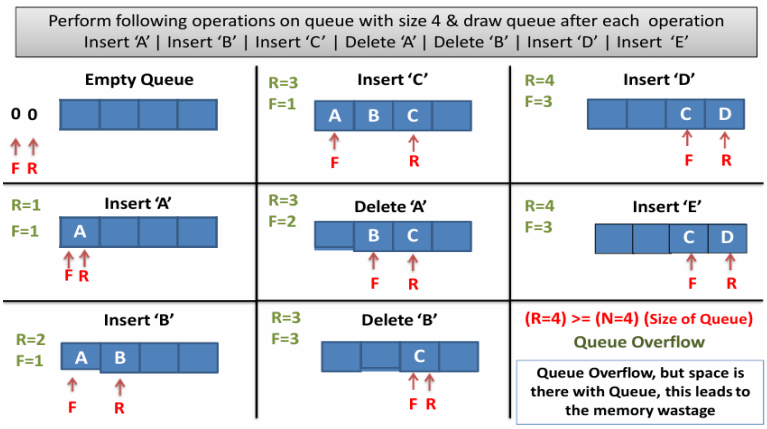
QDELETE(Q,F,R),F=2
Y=Q[2],Y=20,F=3

QDELETE(Q,F,R),F=3
Y=Q[3],Y=30,F=0,R=0
QDELETE(Q,F,R)
F=0 yes then : "Underflow"





Example of Queue Insert / Delete

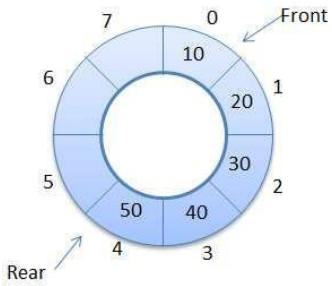


Circular Queue

➢ What is Circular Queue?

- Circular queue is a linear data structure.
- It follows FIFO principle.
- A more suitable method of representing simple queue which prevents an excessive use of memory is to arrange the elements Q[1], Q[2]….,Q[n] in a circular fashion with Q[0] following Q[n], this is called circular queue.
- In circular queue the last node is connected back to the first node to make a circle.





CQINSERT (F, R, Q, N, Y)

- This procedure inserts Y at rear end of the Circular Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

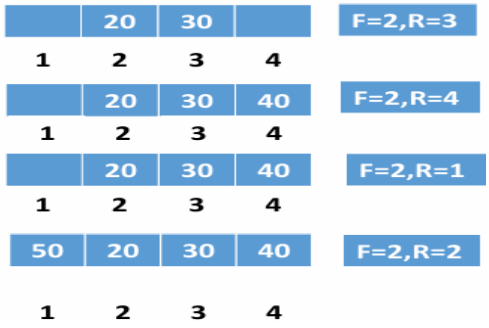
1. [Reset rear pointer?]
   If R = N
   then R ← 1
   else R ← R + 1

2. [Overflow?]
   If F = R
   then Write ('Overflow')
   Return

Queue is having two element, F=2,R=3
CQINSERT(Q,F,R,40) R=N(3=4) False R=R+1=4 F=R(2=4) False, Q[4]=40, F=0, False CQINSERT(Q,F,R,50) R=N(4=4) True ,R=1 F=R(2=1) False, Q[1]=50, F=0, False CQINSERT(Q,F,R,60) R=N(1=4) False R=R+1=2 F=R(2=2) True "Overflow"

3. [Insert element]
   Q[R] ← Y

4. [Is front pointer properly set?]
   If F = 0
   then F ← 1
   Return



CQDELETE (F, R, Q, N)

- This function deletes & returns an element from front end of the Circular Queue.
- Queue is represented by a vector Q containing N elements.
- F is pointer to the front element of a queue.
- R is pointer to the rear element of a queue.

1. [Underflow?]

   If F = 0
   then Write ('Underflow')
   Return (0)

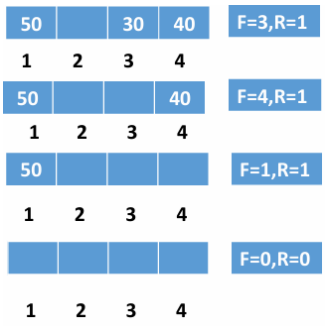
2. [Delete element]
   Y ← Q[F]

3. [Queue empty?]
   If F = R
   then F ← R ← 0
   Return (Y)

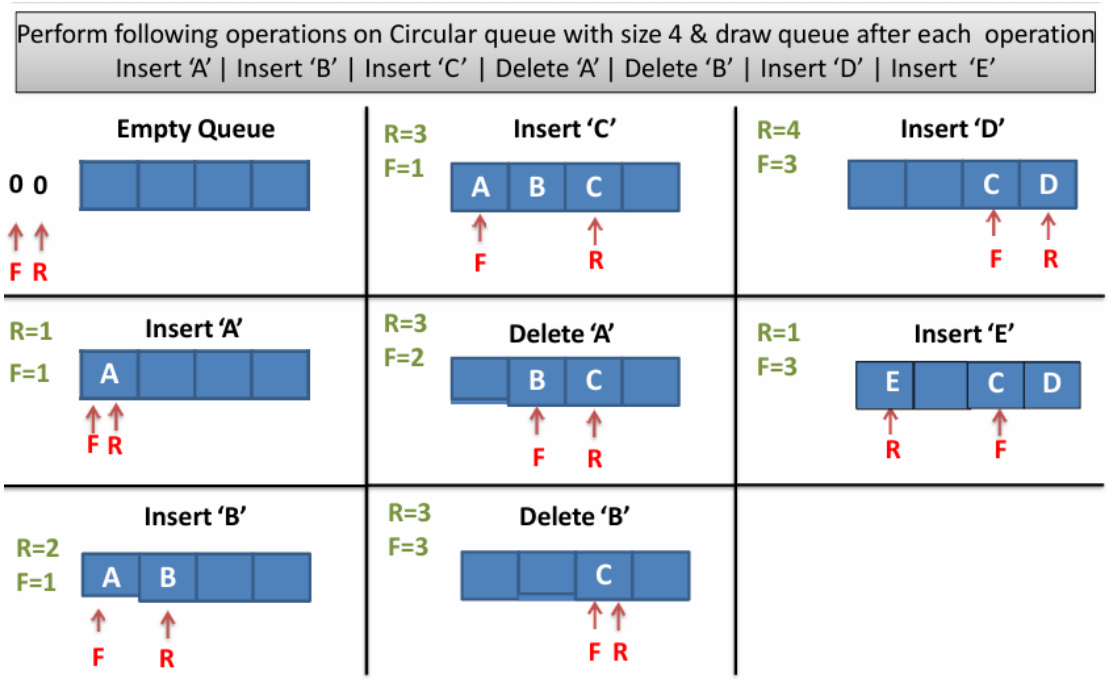
4. [Increment front pointer]
   If F = N
   then F ← 1
   else F ← F + 1
   Return (Y)

Initially 3 element, F=3,R=1 N=4

CQDELETE(Q,F,R,N)
F=0, false,Y=Q[3],Y=30
F=R(3=1) False,
F=N(3=4) false,F=F+1,3+1=4
CQDELETE(Q,F,R,N)
F=0, false,Y=Q[4],Y=40
F=R(4=1) False,
F=N(4=4) True,F=1
CQDELETE(Q,F,R,N)
F=0, false,Y=Q[1],Y=50
F=R(1=1) True,F=0,R=0



Example of Circular Queue Insert / Delete



- **Algorithm: DISPLAY_CircularQueue**

  Input: A circular queue CQ[], front index FRONT, rear index REAR, and size SIZE
  Output: Elements of the circular queue in order from front to rear

  1. IF FRONT == -1
     PRINT "Queue is empty"
     RETURN

  2. SET i = FRONT

  3. REPEAT
     a. PRINT CQ[i]
     b. SET i = (i + 1) % SIZE
  UNTIL i == (REAR + 1) % SIZE

Priority Queue

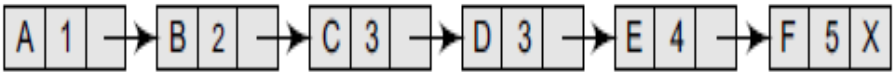
➢ What is a Priority Queue?

- A priority queue is a data structure in which each element is assigned a priority.
- The priority of the element will be used to determine the order in which the elements will be processed

- The general rules of processing the elements of a priority queue are
  - An element with higher priority is processed before an element with a lower priority.
  - Two elements with the same priority are processed on a first-come-first-served (FCFS) basis.
- Priority queues are widely used in operating systems to execute the highest priority process first.

Representation of Priority Queue

- For example, consider the priority queue shown in Fig.

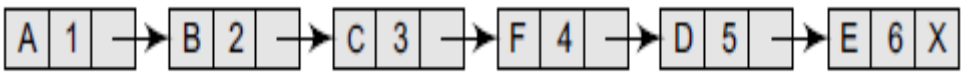


- Lower priority number means higher priority. For example, if there are two elements A and B, where A has a priority number 1 and B has a priority number 5, then A will be processed before B as it has higher priority than B.

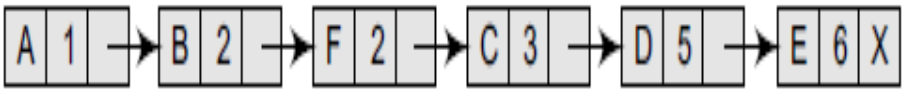
- **Insertion**: When a new element has to be inserted in a priority queue, we have to traverse the entire list until we find a node that has a priority lower than that of the new element.
- The new node is inserted before the node with the lower priority.
- However, if there exists an element that has the same priority as the new element, the new element is inserted after that element.
- For example, consider the priority queue shown in Fig.



- If we have to insert a new element with data = F and priority number = 4, then the element will be inserted before D that has priority number 5, which is lower priority than that of the new element.
- So, the priority queue now becomes as shown in Fig.



- However, if we have a new element with data = F and priority number = 2, then the element will be inserted after B, as both these elements have the same priority but the insertions are done on FCFS basis as shown in Fig.



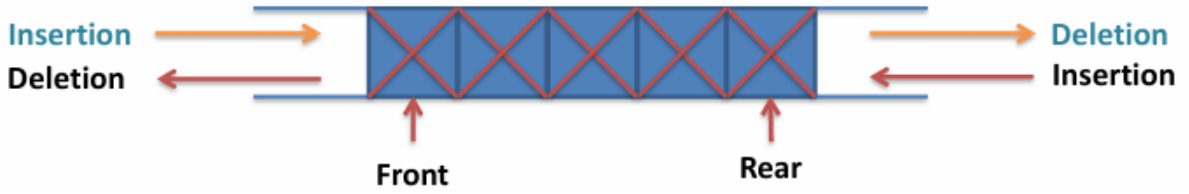
- **Deletion**: Deletion is a very simple process in this case.

- The first node of the list will be deleted and the data of that node will be processed first.

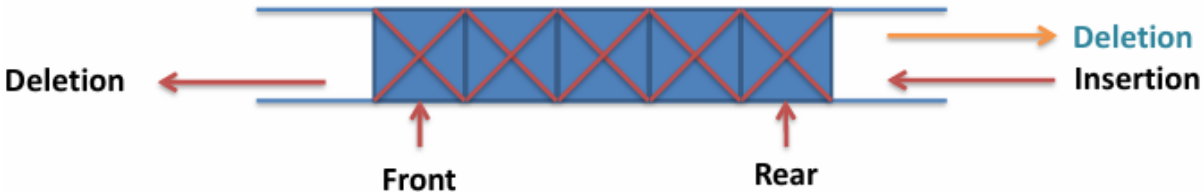
Double Ended Queue

➢ What is Double Ended Queue?

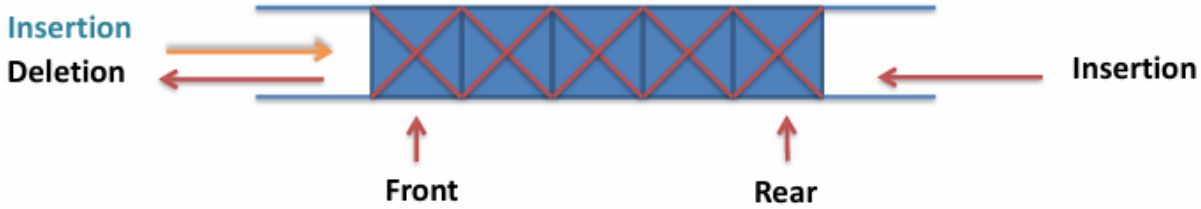
- A deque (pronounced as 'deck' or 'dequeue') is a linear list in which insertion and deletion are performed from the either end of the Queue.
- It is also known as **double ended queue**.



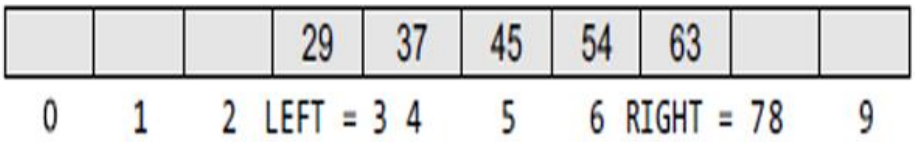
- There are two variations of Dequeue.
- Input restricted dequeue- In this dequeue,insertions can be done only at one of the ends, while deletions can be done from both ends.



- Output restricted dequeue: In this dequeue, deletions can be done only at one of the ends, while insertions can be done on both ends.



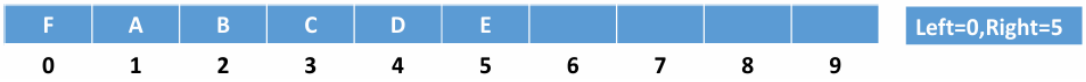
- In a dequeue, two pointers are maintained, LEFT and RIGHT, which point to either end of the dequeue.



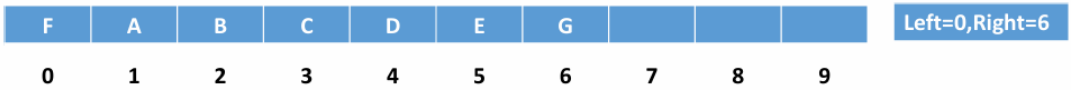
  - **Example Deque: Consider a dequeue given below which has LEFT=1, RIGHT=5**
    _ A B C D E _ _ _ _.
    Now perform the following operations on the dequeue
    1. Add F on the left.
    2. Add G on the right.

3. Add H on the right.
4. Delete two alphabets from left
5. Add I on the right

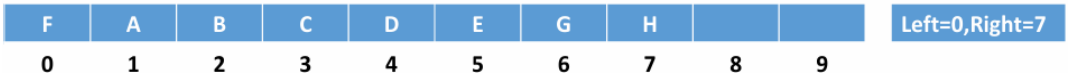
1. Add F on the left.



2. Add G on the right.



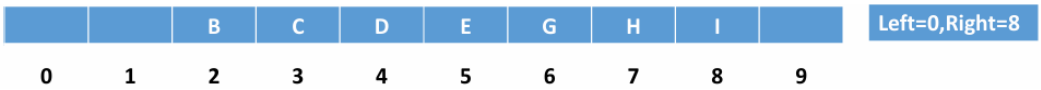
3. Add H on the right.



4. Delete two alphabets from left



5. Add I on the right



Applications of Queue

- Queue of people at any service point such as ticketing etc.
- Queue of air planes waiting for landing instructions.
- Queue of processes in OS.
- Queue is also used by Operating systems for Job Scheduling.
- When a resource is shared among multiple consumers. E.g., in case of printers the first one to be entered is the first to be processed.
- When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.
- Queue is used in BFS (Breadth First Search) algorithm. It helps in traversing a tree or graph.
- Queue is used in networking to handle congestion.

➢ **Difference Between Stack and Queue**

Parameter	Stack Data Structure	Queue Data Structure
Basics	It is a linear data structure. The objects are removed or inserted at the same end.	It is also a linear data structure. The objects are removed and inserted from two different ends.
Working Principle	It follows the Last In, First Out (LIFO) principle. It means that the last inserted element gets deleted at first.	It follows the First In, First Out (FIFO) principle. It means that the first added element gets removed first from the list.
Pointers	It has only one pointer- the **top**. This pointer indicates the address of the topmost element or the last inserted one of the stack.	It uses two pointers (in a simple queue) for reading and writing data from both the ends- the **front** and the **rear**. The rear one indicates the address of the last inserted element, whereas the front pointer indicates the address of the first inserted element in a queue.
Operations	Stack uses **push** and **pop** as two of its operations. The pop operation functions to remove the element from the list, while the push operation functions to insert the element in a list.	Queue uses **enqueue** and **dequeue** as two of its operations. The dequeue operation deletes the elements from the queue, and the enqueue operation inserts the elements in a queue.
Structure	Insertion and deletion of elements take place from one end only. It is called the top.	It uses two ends- front and rear. Insertion uses the rear end, and deletion uses the front end.
Full Condition Examination	When top== max-1, it means that the stack is full.	When rear==max-1, it means that the queue is full.
Empty Condition Examination	When top==-1, it indicates that the stack is empty.	When front = rear+1 or front== -1, it indicates that the queue is empty.
Variants	A Stack data structure does not have any types.	A Queue data structure has three types- circular queue, priority queue, and double-ended queue.

Visualization	You can visualize the Stack as a vertical collection.	You can visualize a Queue as a horizontal collection.
Implementation	The implementation is simpler in a Stack.	The implementation is comparatively more complex in a Queue than a stack.

Programs

- **Operations on Array:**

```
#include <stdio.h>
int main() {
   int A[2][2], B[2][2], C[2][2];

   // Input Matrix A
   printf("Enter elements of Matrix A (2x2):\n");
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         scanf("%d", &A[i][j]);
      }
   }

   // Input Matrix B
   printf("Enter elements of Matrix B (2x2):\n");
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         scanf("%d", &B[i][j]);
      }
   }

   // Addition
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         C[i][j] = A[i][j] + B[i][j];
      }
   }
   printf("\nMatrix Addition (C = A + B):\n");
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         printf("%d\t", C[i][j]);
      }
```

```
        printf("\n");
    }

    // Subtraction
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = A[i][j] - B[i][j];
        }
    }
    printf("\nMatrix Subtraction (C = A - B):\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d\t", C[i][j]);
        }
        printf("\n");
    }

    // Multiplication
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < 2; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    printf("\nMatrix Multiplication (C = A × B):\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d\t", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter elements of Matrix A (2x2):
1
2
3
4
```

```
Enter elements of Matrix B (2x2):
1
2
3
4

Matrix Addition (C = A + B):
2       4
6       8

Matrix Subtraction (C = A - B):
0       0
0       0

Matrix Multiplication (C = A × B):
7       10
15      22
```

- **Factorial:**

```
#include <stdio.h>
int main() {
   int n, i;
   unsigned long long fact = 1;  // use long long for large values

   printf("Enter a number: ");
   scanf("%d", &n);

   if (n < 0) {
      printf("Factorial of a negative number doesn't exist.\n");
   } else {
      for (i = 1; i <= n; i++) {
         fact *= i;
      }
      printf("Factorial of %d = %llu\n", n, fact);
   }

   return 0;
}
```

Output:

```
Enter a number: 5
Factorial of 5 = 120
```

- **Fibonacci using Recursion:**

```
#include <stdio.h>

int fibonacci(int n) {
   if (n == 0) return 0;
   else if (n == 1) return 1;
   else return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
   int n;
   printf("Enter the number of terms: ");
   scanf("%d", &n);

   printf("Fibonacci Series: ");
   for (int i = 0; i < n; i++) {
      printf("%d ", fibonacci(i));
   }
   printf("\n");

   return 0;
}
```

Output:

```
Enter the number of terms: 6
Fibonacci Series: 0 1 1 2 3 5
```

- **Tower of Hanoi problem using recursion:**

```
#include <stdio.h>
void toH(int n, char src, char dest, char intermediate)
{
        if (n == 1)
        {
                printf("\n Move disk 1 from %c to %c",src ,dest );
                return;
        }
        toH(n-1, src, intermediate, dest);
        printf("\n Move disk %d from %c to %c", n, src, dest);
```

```
        toH(n-1,intermediate,dest,src);
}

int main()
{
        int no_of_disks ;
        printf("Enter number of disks: ");
        scanf("%d", &no_of_disks);
        toH(no_of_disks, 'A','C','B');
        return 0;
}
```

Output:

```
Enter number of disks: 3

 Move disk 1 from A to C
 Move disk 2 from A to B
 Move disk 1 from C to B
 Move disk 3 from A to C
 Move disk 1 from B to A
 Move disk 2 from B to C
 Move disk 1 from A to C
```

- **Stack Operations:**

```
#include <stdio.h>
#define MAX 5   // Maximum size of stack
int stack[MAX], top = -1;

// Function to push an element onto the stack
void push(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow! Cannot push %d\n", value);
   } else {
      top++;
      stack[top] = value;
      printf("%d pushed to stack\n", value);
   }
}
// Function to pop an element from the stack
void pop() {
```

```
    if (top == -1) {
        printf("Stack Underflow! Nothing to pop\n");
    } else {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}
// Function to display stack elements
void display() {
    if (top == -1) {
        printf("Stack is empty!\n");
    } else {
        printf("Stack elements: ");
        for (int i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
// Function to peep (view) element at a given position from top
void peep(int pos) {
    int index = top - pos + 1;   // position from top
    if (index < 0 || index > top) {
        printf("Invalid position! Cannot peep.\n");
    } else {
        printf("Element at position %d from top is: %d\n", pos, stack[index]);
    }
}
// Function to change element at a given position from top
void change(int pos, int value) {
    int index = top - pos + 1;
    if (index < 0 || index > top) {
        printf("Invalid position! Cannot change.\n");
    } else {
        printf("Element %d at position %d changed to %d\n", stack[index], pos, value);
        stack[index] = value;
    }
}
int main() {
    int choice, value, pos;

    while (1) {
        printf("\n--- Stack Menu ---\n");
```

```
        printf("1. Push\n2. Pop\n3. Display\n4. Peep\n5. Change\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Enter position from top to peep: ");
                scanf("%d", &pos);
                peep(pos);
                break;
            case 5:
                printf("Enter position from top to change: ");
                scanf("%d", &pos);
                printf("Enter new value: ");
                scanf("%d", &value);
                change(pos, value);
                break;
            case 6:
                return 0;
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
}
```

Output:

```
--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
```

```
5. Change
6. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 1
Enter value to push: 30
30 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 3
Stack elements: 30 20 10

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
```

5. Change
6. Exit
Enter your choice: 4
Enter position from top to peep: 2
Element at position 2 from top is: 20

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 5
Enter position from top to change: 2
Enter new value: 50
Element 20 at position 2 changed to 50

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 4
Enter position from top to peep: 2
Element at position 2 from top is: 50

--- Stack Menu ---
1. Push
2. Pop
3. Display
4. Peep
5. Change
6. Exit
Enter your choice: 2
30 popped from stack

--- Stack Menu ---
1. Push
2. Pop
3. Display

4. Peep
5. Change
6. Exit
Enter your choice: 6

- **Queue Operations:**

```
#include <stdio.h>
#define MAX 5

int queue[MAX], front = -1, rear = -1;

// Function to insert (enqueue) an element
void enqueue(int value) {
    if (rear == MAX - 1) {
        printf("Queue Overflow! Cannot insert %d\n", value);
    } else {
        if (front == -1)  // first insertion
            front = 0;
        rear++;
        queue[rear] = value;
        printf("%d inserted into queue\n", value);
    }
}

// Function to delete (dequeue) an element
void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow! Nothing to delete\n");
    } else {
        printf("%d deleted from queue\n", queue[front]);
        front++;
    }
}

// Function to display elements of queue
void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty!\n");
    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
```

```
        }
        printf("\n");
    }
}

int main() {
    int choice, value;
    while (1) {
        printf("\n--- Queue Menu ---\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
}
```

Output:

```
--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 30
30 inserted into queue
```

```
--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 40
40 inserted into queue

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 50
50 inserted into queue

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 30 40 50

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
30 deleted from queue

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 40 50

--- Queue Menu ---
```

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 40 50

--- Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
```

- **Circular Queue Program:**

```
#include<stdio.h>
# define MAX 3
int cqueue_arr[MAX];
int front = -1;
int rear = -1;
void insert(int item)
{
if((front == 0 && rear == MAX-1) || (front == rear+1))
{
printf("Queue Overflow\n");
return;
}
if(front == -1)
{
front = 0;
rear = 0;
}
else
{
if(rear == MAX-1)
rear = 0;
else
rear = rear+1;
}
cqueue_arr[rear] = item ;
}
void deletion()
```

```
{
if(front == -1)
{
printf("Queue Underflow");
return ;
}
printf("Element deleted from queue is : %d\n",cqueue_arr[front]);
if(front == rear)
{
front = -1;
rear=-1;
}
else
{
if(front == MAX-1)
front = 0;
else
front = front+1;
}
}
void display()
{
int i;
for(i=front;i!=rear;i=(i+1)%MAX)
{
printf("%d\t",cqueue_arr[i]);
}
printf("%d",cqueue_arr[rear]);
}
int main()
{
int choice,item;
do
{
printf("\n1.Insert \n");
printf("2.Delete\n");
printf("3.Display\n");
printf("4.Quit\n");
printf("Enter your choice : ");
scanf("%d",&choice);
switch(choice)
{
case 1 :
```

```
printf("Input the element for insertion in queue : ");
scanf("%d", &item);
insert(item);
break;
case 2 :
deletion();
break;
case 3:
display();
break;
case 4:
break;
default:
printf("Wrong choice ");
}
}while(choice!=4);
return 0;
}
```

Output:

```
1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 1
Input the element for insertion in queue : 10

1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 1
Input the element for insertion in queue : 20

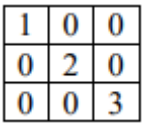
1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 1
Input the element for insertion in queue : 30

1.Insert
2.Delete
```

```
3.Display
4.Quit
Enter your choice : 3
10      20      30
1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 2
Element deleted from queue is : 10

1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 3
20      30
1.Insert
2.Delete
3.Display
4.Quit
Enter your choice : 4
```

GTU Questions from Unit 2

Sr. No	Questions	CO	Answer	Marks
1	What is Sparse matrix? Write efficient vector representation of following Sparse matrix. [NLJIET]  What is Sparse Matrix? [NLJIET]	2	Pg 10,11	3
2	What is stack? Why do we use multiple stacks? [NLJIET]	2	Pg 11,12	3
3	Illustrate how stack is used in the recursion. [NLJIET]	2	Pg 11,12	3
4	Write applications of stack and queue data structures. [NLJIET] Enlist and explain any 3 applications of stack in computer science. [NLJIET]	2	Pg 14,15, 35	3
5	Write an algorithm for infix to postfix conversion. [NLJIET]	2	Pg 16,17	3
6	Evaluate the following postfix expression using stack. (i) 53+62/*35*+ [NLJIET]	2	Pg 21	3
7	Explain Tower of Hanoi with suitable example. [NLJIET] Write a recursive solution for Tower of Hanoi problem.	2	Pg 25,26	3
8	Distinguish between stack and queue. [NLJIET]	2	Pg 35,36	3
9	Explain advantages of circular queue over Simple queue. [NLJIET]	2	Pg 29	3
10	Explain the working of priority queue with suitable example. [NLJIET]	2	Pg 32,33	3
11	What is priority queue? Is simple queue is anyhow priority queue? Explain your answer. [NLJIET] What is priority queue? [NLJIET]	2	Pg 32,33	3
12	How to prove array is sequential and contiguous? [NLJIET]	2	Pg 2	4
13	Explain row-major order and column-major order	2	Pg 5	4

	representation of 2-D array. [NLJIET]			
14	State at least one efficient representation of a sparse matrix. [NLJIET]	2	Pg 10,11	4
15	Derive basic operation of stack and write C function to implement it. [NLJIET]	2	Pg 12,13,14	4
16	What is top of stack? Why stack is called LIFO list? [NLJIET]	2	Pg 12	4
17	Write an algorithm to evaluate postfix expression. Explain working of the algorithm using appropriate example. [NLJIET]	2	Pg 20,21	4
18	Convert the following infix expressions to their prefix and postfix equivalents. (i) (A*B)+(C/D)-(D+E) [NLJIET] Convert the following infix expression into postfix expression using stack. (i) (A-B)/C*D^(E/F)^(G+H) [NLJIET] Convert the following infix expressions to their prefix. (i) (A^B*C-D+E/F/(G+H)) [NLJIET] Translate infix expression into its equivalent post fix expression: A*(B+D)/E-F*(G+H/K) [NLJIET]	2	Pg 18	4
19	Write C program to find the Fibonacci sequence of n terms using recursion. [NLJIET]	5	Pg 42	4
20	Explain Tower Of Hanoi with example. [NLJIET] Write recursive solution for tower of Hanoi. How many moves require for transferring three discs? [NLJIET]	2	Pg 25,26	4
21	State disadvantages of simple queue. How to overcome it? [NLJIET] Write algorithm to insert into simple queue and mention the limitation of simple queue? [NLJIET]	2	Pg 26,27,28	4
22	Design an algorithm to perform insert operation in circular queue. [NLJIET]	2	Pg 29, 30	4
23	Write algorithm to delete from circular queue and mention the advantage of circular queue over simple queue? [NLJIET]	2	Pg 29,30,31	4
24	Explain Dequeue and Priority queue in detail. [NLJIET]	2	Pg 32,33, 34,35	4
25	Explain Row Major and Column Major with	2	Pg	7

	Example. [NLJIET]		5,6,7	
26	Write and explain algorithms for PUSH and POP operations on stack data structure. [NLJIET] Define stack. Write algorithms for PUSH, POP and PEEP operations of stack. [NLJIET]	2	Pg 12,13,14	7
27	Write a recursive function to compute factorial of a number. Show usage of STACK in recursion for this function. [NLJIET]	5	Pg 41	7
28	What is stack? Explain operations on stack in detail. [NLJIET]	2	Pg 11,12, 13,14	7
29	Write an algorithm for the following stack operations. 1) PUSH 2) POP 3) DISPLAY [NLJIET]	2	Pg 11,12, 13,14	7
30	Evaluate the following postfix expression in tabular form: (i) 5 * 6 2 / + [NLJIET] (ii)8 2 / 6 7 * + [NLJIET] (iii) 2 5 3 - * 8 / 4 + [NLJIET] Evaluate the prefix expression in tabular forms: (i) / - 8 2 3 4 [NLJIET] (ii)/ 7 * 1 + 4 – 6 3 [NLJIET] Explain Tower Of Hanoi with example [NLJIET] Explain Sparse matrix with example. [NLJIET]	2	Pg 21,25, 26,10,11	7
31	Convert following Infix expression into Postfix and Prefix expression. (i) (A*B)+(C/D)-(D+E) [NLJIET] Convert the following infix expressions to prefix expressions and show the stack trace. (i) A*(B+C)/D-E*F^G (ii) A+B*(C-D/E$F)*G [NLJIET]	2	Pg 18	7
32	Convert following infix expression into postfix expression using stack. (i) (A-B)/C*D^(E/F)^(G+H) [NLJIET] Convert following infix expressions to the postfix expressions. Shows stack trace. (i) ((A/(B-C+D))*(E-F)*G) [NLJIET] Convert the following infix expressions to their prefix. (i) (A^B*C-D+E/F/(G+H)) [NLJIET] Convert the following infix expression into a postfix expression using stack.	2	Pg 18	7

	(i) (a+b)^((c*d)/(e-f)) [NLJIET] Write an algorithm to convert infix expression into postfix expression. [NLJIET]			
33	Evaluation the following expression showing every status of stack in tabular form. (i) 3 5 * 6 2 / + [NLJIET] Evaluate the following postfix expression using a stack. Show the stack contents. (i) 53+62/*35*+ [NLJIET] Write an algorithm to evaluate postfix expression. Explain working of the algorithm using appropriate example. [NLJIET] Evaluate the following postfix expression and show the stack trace. 6 5 1 – 4 * 2 3 ^ / + [NLJIET]	2	Pg 21	7
34	Explain the concept of Reverse Polish Notation (RPN) and describe the process for evaluating an RPN expression using a stack. Provide a detailed example of evaluating the following RPN expression: 5 1 2 + 4 * + 3 - [NLJIET]	2	Pg 16,17,21	7
35	Write a ‘C’ program to reverse a string using stack. [NLJIET]	5	Extra Programs	7
36	What is queue? Explain operations on queue in detail. [NLJIET]	2	Pg 26,27,28	7
37	What is problem with simple queue? Explain its solution with example and algorithms. [NLJIET]	2	Pg 26,27,28	7
38	Write an algorithm to perform insert and delete operations on simple queue. [NLJIET]	2	Pg 27,28	7
39	Write algorithms to insert, and delete elements in queue. [NLJIET]	2	Pg 27,28	7
40	Define queue. Write algorithms for INSERT, DELETE and DISPLAY operations of queue. [NLJIET] Write algorithm to (i) insert, and (ii) delete elements in circular queue. [NLJIET] Write an algorithm for the following queue operations. 1) INSERT 2) DELETE 3) DISPLAY [NLJIET]	2	Pg 26,27,28	7
41	Write a C program to implement simple queue and check for boundary conditions. [NLJIET]	5	Pg 47,48, 49,50	7
42	Discuss and write a program to implement queue	5	Pg	7

	functions using arrays. [NLJIET]		47,48, 49,50	
43	What is a circular queue? How do you check the queue full condition? Write an algorithm to count the nodes in a circular queue. [NLJIET]	2	Pg 29,30	7
44	Write a program to implement Circular queue and show how it differ from normal queue? [NLJIET]	5	Pg 50,51, 52,53	7
45	Write an algorithm for INSERT, DELETE and DISPLAY function of Circular Queue. [NLJIET]	2	Pg 29,30,31,	7
46	Write a C program for the following operations on a circular queue. 1. Insert 2. Delete 3. Display [NLJIET] Write a C program to implement circular queue with insertion and deletion operations. [NLJIET]	5	Pg 50,51, 52,53	7
47	What is priority queue? Discuss its applications and implementation details. [NLJIET]	2	Pg 32,33	7